Rapport d'Audit de Sécurité - Challenge UnCrackable Level 2

Auteur : Sultan Zeineddine

Établissement : École Nationale des Sciences Appliquées (ENSA), Marrakech

Spécialité : Cyber Défense et Systèmes de Télécommunications Embarqués

Date : 24 Avril 2026

1. Introduction

Ce rapport documente l'audit de sécurité et la rétro-ingénierie effectués sur l'application UnCrackable-Level2.apk. Ce challenge, issu de l'OWASP Mobile Security Testing Guide (MSTG), vise à tester les compétences en analyse de bibliothèques natives Android, une technique souvent utilisée pour dissimuler des algorithmes sensibles ou des secrets.

2. Analyse Statique Préliminaire (Java)

La première phase de l'audit a consisté en une décompilation du bytecode Dalvik à l'aide de l'outil JADX-GUI. L'objectif était d'identifier le mécanisme de validation de la phrase secrète.

2.1 Détection de la couche native

L'analyse du code source Java a révélé que la validation ne s'effectue pas dans la couche Java. La classe CodeCheck fait appel à une méthode native :

```
public native boolean bar(byte[] bArr);
```

L'utilisation de System.loadLibrary(\"foo\") confirme que la logique est encapsulée dans un fichier binaire nommé libfoo.so. À ce stade, les outils de décompilation Java conventionnels ne permettent plus d'analyser le flux de validation.

3. Extraction et Préparation de la Bibliothèque Native

Pour poursuivre l'analyse, l'APK a été décompressé afin d'accéder au dossier lib/. Plusieurs architectures étaient disponibles :

- *arm64-v8a / armeabi-v7a*
- *x86 / x86_64*

L'architecture x86 a été sélectionnée pour l'analyse sous Windows, permettant l'utilisation du fichier libfoo.so dans un environnement de rétro-ingénierie binaire.

4. Analyse Binaire avec Ghidra

Le framework de rétro-ingénierie Ghidra a été utilisé pour analyser le binaire ELF. La recherche s'est concentrée sur le symbole Java_sg_vantagepoint_uncrackable2_CodeCheck_bar.

4.1 Correction du flux d'exécution

Lors de l'analyse initiale, Ghidra a rencontré des instructions malformées (halt_baddata), une technique d'anti-désassemblage volontairement implémentée pour tromper les analystes. La correction a nécessité les étapes suivantes :

1. *Identification des octets de code brut dans la vue Listing.*
2. *Utilisation de l'instruction Disassemble (D) pour forcer la lecture du langage machine.*
3. *Utilisation de Create Function (F) pour reconstruire le graphe de contrôle de flux.*

4.2 Identification des Stack Strings

Le pseudo-code généré a révélé que le secret est construit dynamiquement sur la pile sous forme de blocs hexadécimaux de 32 bits. Cette technique empêche la recherche de chaînes de caractères en clair via la commande strings.

Variable Pile	*Valeur Hexadécimale*	*Interprétation ASCII*
uStack_30	*0x6e616854*	*Than*
uStack_2c	*0x6620736b*	*ks f*
uStack_28	*0x6120726f*	*or a*
uStack_24	*0x74206c6c*	*ll t*
uStack_20	*0x6568*	*he*
uStack_1e	*0x73696620*	*fis*

5. Règle du Little Endian et Décodage Final

L'architecture x86 étant Little Endian, les octets sont stockés de l'octet de poids faible à l'octet de poids fort. Pour retrouver le texte original, il a fallu lire chaque groupe d'octets de droite à gauche.

Exemple pour 0x6e616854 :

Octets : 54 68 61 6e → ASCII : Than

La phrase secrète complète reconstituée est : \"Thanks for all the fish\".

6. Automatisation

Un script Python a été développé pour automatiser le processus de conversion et valider la logique de reconstruction.

```
# Script de décodage Little Endian
hex_vals = [\"6e616854\", \"6620736b\", \"6120726f\", \"74206c6c\", \"6568\", \"73696620\", \"68\"]
password = \"\".join([bytes.fromhex(h)[::-1].decode() for h in hex_vals])
```

print(password)

7. Conclusion

Ce lab a démontré que la sécurité par l'obscurité (via JNI et les Stack Strings) peut ralentir une analyse mais ne peut empêcher un auditeur expérimenté d'extraire des secrets. En tant que futurs ingénieurs en Cyber Défense, il est recommandé de ne jamais stocker de secrets en dur dans le code client, même dans les couches natives.